



Feature, Not a Bug

Sprint #2 Review

ALEX HANUS

JARRET FOSSEN

JOSE ORDONEZ

AMARI THOMAS

OSBALDO OROZCO

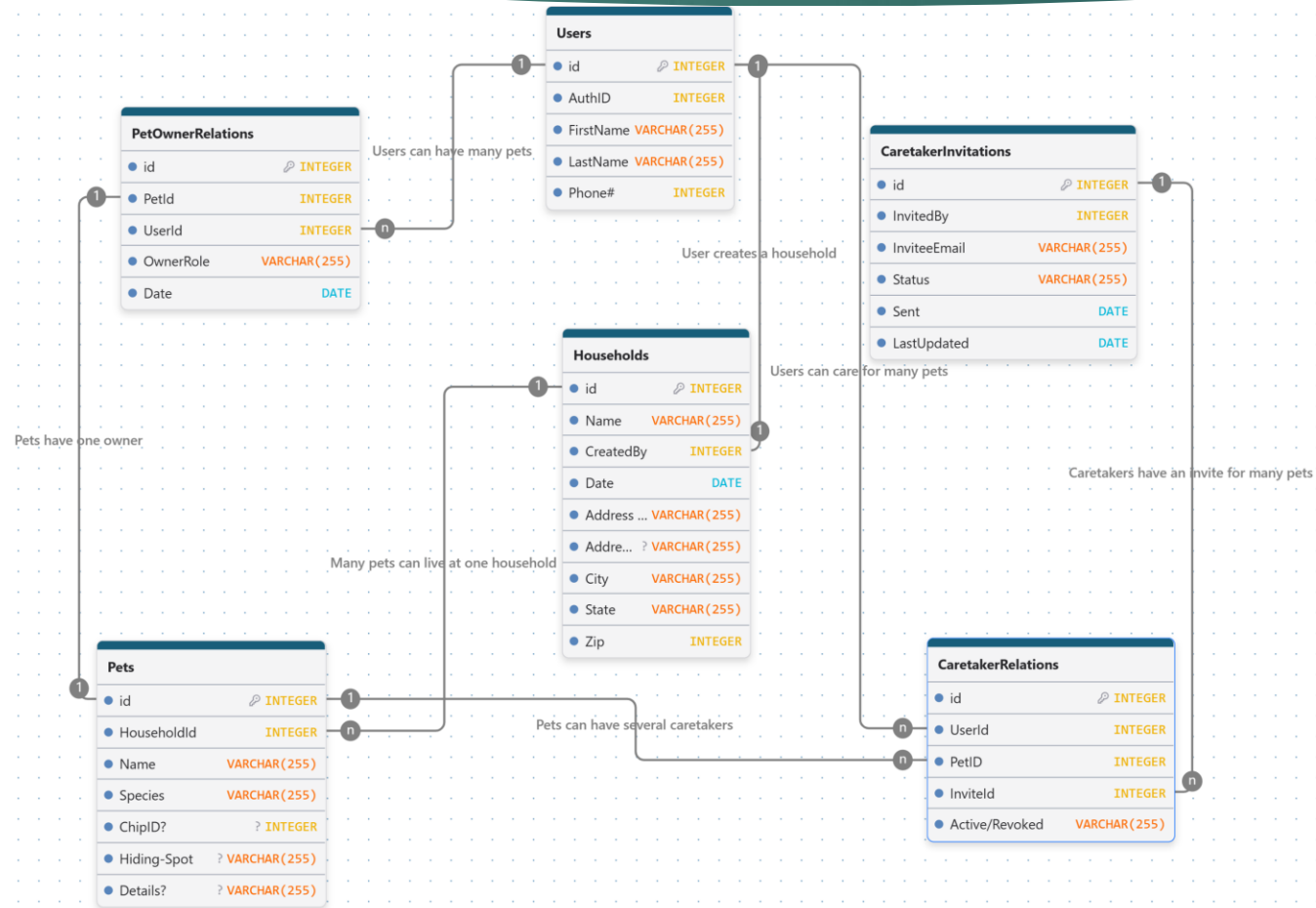
Sprint #2 Goal

Build a working web pet-profile flow that allows a user to create multiple dogs or cats, save them locally, return later to view their pets, and open a separate detail page for each pet. Prepare the backend foundation for eventual authenticated PostgreSQL persistence. Connecting the web flow to the backend and porting it to mobile are stretch goals.

Sprint #2 backlog: What We Planned and Completed

September 25 – October 6, 2026		Total time logged: 25h 5m	
Work item	Points	Time logged	Sprint 2 result
Set up PostgreSQL access and migrations in NestJS	4.5	3h 45m	Completed
Set up web pet routes and local pet storage	3	3h 30m	Completed
Build Create Pet page with local saving	2	—	Completed
Build My Pets page showing saved pets	2	—	Completed
Build Pet Details page for a saved pet	2	—	Completed
Design the core ReadyPets database schema	3.5	3h 20m	Completed
Sprint 2 backlog refinement and Jira organization	0	2h 30m	Completed
Updated the CSS for My Pets and Create Pets	2	—	Completed
Build a basic public homepage	1	30m	Completed
Set up safe, repeatable backend database testing	4.5	3h 30m	Completed
Connect Google sign-in to the authenticated ReadyPets account flow	4	5h 30m	Completed
Basic research regarding future services to consider	1	1h 30m	Completed
Configure automated testing for the web pet-profile flow	3	1h	Moved to Sprint 3
Connect the web pet-profile flow to authenticated, owner-authorized APIs	2	—	Moved to Sprint 3
Completed: 12 tasks / 29.5 points		Carried into Sprint 3: 2 tasks / 5 points	
Includes work added during Sprint 2.		— = no time logged in Jira.	

Schema Diagram



Tech Decisions this Sprint

- ▶ Type ORM

- ▶ Works alongside PostgreSQL to make database access and schema changes easier to manage from NestJS

- ▶ Docker

- ▶ To run a disposable, local Supabase environment for database testing
 - ▶ Allows us to test database API without touching the real database

PostgreSQL Setup & Migrations

- ▶ Goal: Connect the NestJS backend to the Supabase PostgreSQL database
 - ▶ Connected the NestJS backend to the Supabase PostgreSQL database
 - ▶ Used TypeORM + pg for database access and migrations
 - ▶ Created the initial database schema with tables and foreign-key relationships
 - ▶ Kept database credentials secure in a backend .env file
 - ▶ Successfully ran the migration and passed CI checks

PostgreSQL Setup & Migration

Supabase – the online service we use to host and manage our project's database

PostgreSQL – database system that stores information

TypeORM – tool that helps our backend communicate with the database without us having to manually write everything from scratch

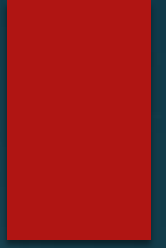
Pg – The connector that lets our backend talk specifically to a PostgreSQL database

.env - A private configuration file where we keep sensitive info

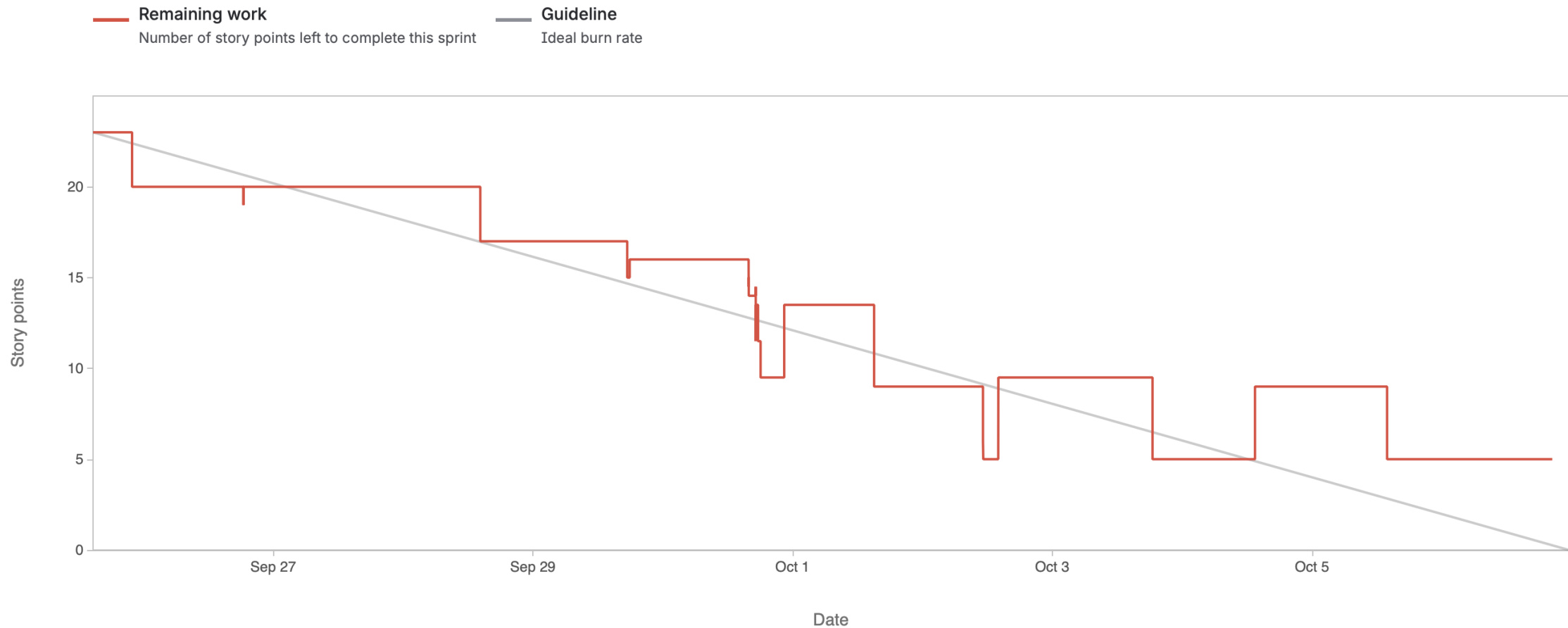
Migration – A controlled way to create or update the database structure

CI Checks – Automatic tests that run when we submit code to make sure nothing is broken before it gets merged

DEMO

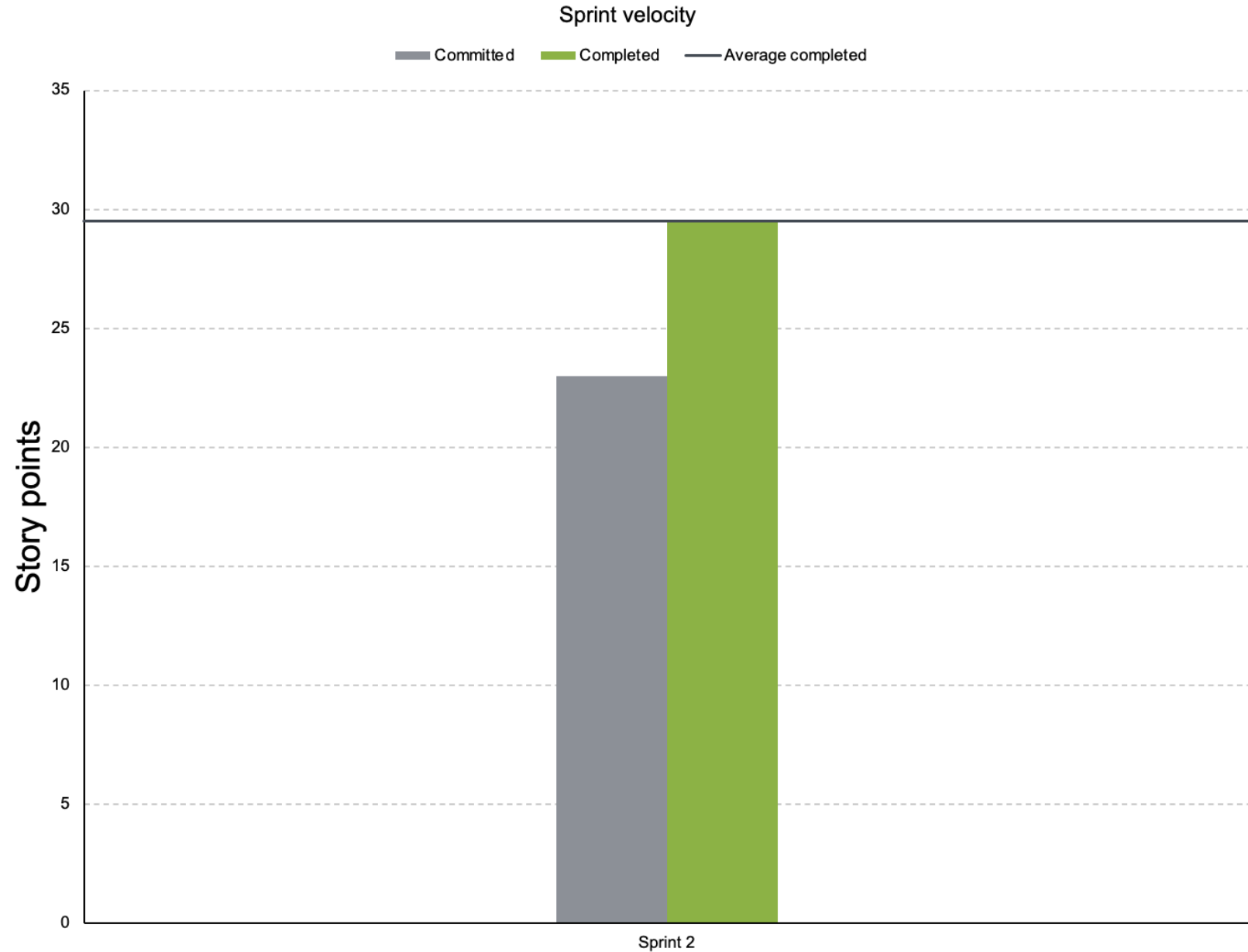


Sprint #2 Burndown



Sprint #2 Velocity

- Sprint 1 is excluded from the chart and average due to insufficient data.
- Sprint 2 will serve as the baseline for velocity



Sprint #2 Retrospective

What went well

- ▶ Resolved major foundational blockers: database schema design, database setup, and secure Google sign-in
- ▶ Tested the implemented functionality throughout the sprint
- ▶ Cleared dependencies that will allow more parallel development next sprint

What we will improve

- ▶ Task ownership: Clarify responsibilities and expectations so everyone can contribute effectively next sprint
- ▶ Communication: Share progress regularly and flag difficulties when help is needed
- ▶ Parallel development: Divide mobile app and API work into tasks teammates can advance concurrently

Sprint #3 Plan

Main Goal

Build a working ReadyPets mobile app where users can sign in, add dogs or cats, and view their saved pet profiles. Connect it to the shared API so pets are saved in the database and users can only access their own pets.

Task	Story points
Build account address APIs	5
Build pet creation, listing, and details APIs	5
Build pet editing and deletion APIs	3
Finish connecting the web pet pages to the API	From Sprint 23
Finish automated testing for the web pet pages	From Sprint 22
Implement Google sign-in and sessions on mobile	5
Build the mobile layout and navigation	2
Build mobile account and address setup	3
Build the mobile Create Pet screen	3
Build mobile My Pets and Pet Details screens	3
Add mobile pet editing and deletion	3
Research Apple sign-in and deployment requirements	2
Planned total: 12 tasks	39
Stretch goals — outside the 39-point plan	
Continuous deployment: 5 points • Apple sign-in: 5 points	